Arming Data Agents with Tribal Knowledge

Shubham Agarwal[†], Asim Biswal[†], Sepanta Zeighami[†]
Alvin Cheung, Joseph Gonzalez, Aditya G. Parameswaran
UC Berkeley
{shubham3,abiswal,zeighami,akcheung,jegonzal,adityagp} @berkeley.edu

ABSTRACT

Natural language to SQL (NL2SQL) translation enables non-expert users to query relational databases through natural language. Recently, NL2SQL agents, powered by the reasoning capabilities of Large Language Models (LLMs), have significantly advanced NL2SQL translation. Nonetheless, NL2SQL agents still make mistakes when faced with large-scale real-world databases because they lack knowledge of how to correctly leverage the underlying data (e.g., knowledge about the intent of each column) and form *misconceptions* about the data when querying it, leading to errors. Prior work has studied generating facts about the database to provide more context to NL2SQL agents, but such approaches simply restate database contents without addressing the agent's misconceptions. In this paper, we propose Tk-Boost, a bolt-on framework for augmenting any NL2SQL agent with *tribal knowledge*: knowledge that corrects the agent's misconceptions in querying the database accumulated through experience using the database. To accumulate experience, Tk-Boost first asks the NL2SQL agent to answer a few queries on the database, identifies the agent's misconceptions by analyzing its mistakes on the database, and generates tribal knowledge to address them. To enable accurate retrieval, Tk-Boost indexes this knowledge with *applicability conditions* that specify the query features for which the knowledge is useful. When answering new queries, Tk-Boost uses this knowledge to provide feedback to the NL2SQL agent, resolving the agent's misconceptions during SQL generation, and thus improving the agent's accuracy. Extensive experiments across the BIRD and Spider 2.0 benchmarks with various NL2SQL agents shows **Tk-Boost improves NL2SQL agents accuracy by up to 16.9% on Spider 2.0 and 13.7% on BIRD**.

1 INTRODUCTION

Natural language (NL) interfaces to databases enable users without SQL expertise to query data in relational databases using natural language. In recent years, the proficiency of Large Language Models (LLMs) in writing SQL, coupled with tool-calling capabilities [16, 50] has led to significant breakthroughs in accurate NL2SQL translation [11, 21, 28, 40]. These *NL2SQL agents* accept NL queries and return SQL, and vary across prompting strategies, provided tools (e.g., database or file system access), and workflows they are embedded in (e.g., existence of validation loops or consistency mechanisms [21, 35, 44]).

Limitations of NL2SQL Agents. Despite significant improvements in recent years, agents still make mistakes in large real-world databases, with many columns, tables, and rows. This is due to gaps in the agent's knowledge about databases not seen during training set—such as its contents, how it was created and its intended use. Such knowledge gaps, especially about subtle data idiosyncrasies, cause logical errors in translation [10, 29], where agents use incorrect data sources (tables or columns) or perform logically incorrect operations on them. We refer to these recurring logical errors in performing queries caused by the agent's knowledge gaps as *misconceptions*. For example, multiple tables or columns with similar semantics are common in data lakes, where derived tables with modified columns (e.g., NULLs removed through imputation), are stored alongside the originals, causing misconceptions when deciding which column to use. We use such a scenario as our running example: a table containing product sales information with columns `name` and `brand`, with `brand` inconsistently populated (perhaps because it was added later on), while `name` reliably contains both the product name and brand, with the latter as a prefix (e.g., "Nike Air Max"). An agent using `brand` for queries such as "find average sales by brand" produces incorrect SQL, with errors recurring for any query that requires product brand information.

Prior work has tried to fix misconceptions by proactively identifying "facts", typically by asking LLMs to generate them about the database content [3, 5, 31, 51, 52]. However, doing so in the absence of queries simply restates this content without fixing the misconceptions that arise when answering queries. In our example, one fact may be that both `name` and `brand` columns contain product brands, but this information does not help the agent infer that `name` should be used instead of the `brand` column.

Agents with Tribal Knowledge. We propose Tk-Boost, a framework for correcting NL2SQL agents' misconceptions. Our key insight is that an NL2SQL agent should accumulate knowledge of how to correctly query a database *through experience*, rather than relying on facts derived from the data—this process is analogous to how data analysts or engineers accumulate knowledge over time through exposure to the database. A simple approach to accumulate experience is to store the agent's interactions with the database (i.e., queries and answers)—or their summaries—and retrieve relevant ones to include in the agent's prompts for future queries via a form of RAG (retrieval augmented generation) using LLM memory stores [24, 25, 32, 42, 45, 49] such as Mem0 [32].

However, as we show, directly including past interactions does not correct *misconceptions* as past mistakes end up being repeated. Instead, Tk-Boost augments agents with what we call *tribal knowledge*: natural language statements that *correct the agent's misconceptions* and can be *reused to correctly answer related future queries* (e.g., stating "use `name` column to find product brands instead of `brand`" in our running example). To enable an agent to accumulate tribal knowledge through experience, Tk-Boost uses a few labeled query examples (i.e., NL queries with ground-truth SQL) and asks the agent to answer those queries on the database. Tk-Boost then *discovers tribal knowledge* from its experience and *uses*

[†]Equal contribution in alphabetical order.

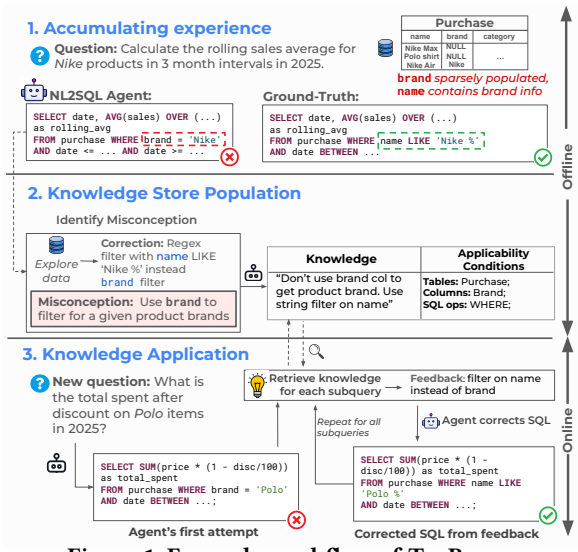


Figure 1: Example workflow of Tk-Boost.

the knowledge to improve the agent's accuracy for future queries. Operationalizing these two steps is hard as it requires identifying misconceptions, generating reusable knowledge that fixes them, and using it correctly for future queries, as we discuss next.

Tribal Knowledge Discovery. Tk-Boost identifies agent's misconceptions through analyzing the agent's *incorrect SQL queries* and generates tribal knowledge—reusable natural language statements that correct the misconceptions that lead to incorrect queries. Accurately identifying misconceptions is challenging because many SQL queries contain complex logic that are difficult to comprehend, while errors depend on data idiosyncrasies so detecting them require an understanding of the underlying data. Simply prompting an LLM to derive knowledge by comparing the agent-generated and ground-truth SQL (similar to summarizing past experience in memory stores [4, 9, 25]) often yields incorrect or trivial statements because the LLM lacks context about database and the complexity of queries cause mistakes. Additionally, future queries are often very different from the small set of past queries the knowledge is generated from. Knowledge statements naively generated with an LLM, even when correct, often contains query-specific knowledge that cannot help address misconceptions for future queries.

To reliably discover tribal knowledge, Tk-Boost tries to identify the root cause of the error through iteration on the query and the database. It iteratively makes small data-aware modifications—changing one SQL clause at a time—to the incorrect agent-generated SQL until it produces the correct answer, and detects logical errors along the way. Tk-Boost then generates tribal knowledge statements in NL for each logical error. Each statement explains how to generate a correct SQL not only for the NL query where the logical error was identified, but more broadly to other queries where similar errors may appear, the latter achieved through proactively identifying such queries and generating knowledge statements that correct errors across them. Fig. 1 illustrates this process. The NL2SQL agent accumulates experience by performing a query for which it incorrectly uses the `brand` column (Step 1). Tk-Boost identifies the agent's misconception and generates knowledge that `name` should be used to obtain product brand information (Step 2).

Tribal Knowledge Usage. For a new NL query, Tk-Boost retrieves relevant knowledge statements and uses it to provide feedback to improve the agent's accuracy. Each statement corrects a specific misconception and must be retrieved and applied for queries where the misconception occurs. Doing so accurately is difficult because misconceptions often occur when performing fine-grained sub-tasks required for NL queries (e.g., when joining with a specific table), which are not directly stated in NL queries that are more coarse-grained (e.g., don't mention a join on this table). Thus, naively retrieving knowledge using the NL query—for example, through embedding similarity—yields limited gains because it cannot correctly identify the fine-grained sub-tasks needed for the query. Additionally, when given the relevant knowledge statements, agents often incorrectly use them for sub-tasks they do not apply to, especially for complex queries with many relevant knowledge statements. Thus, simply injecting all knowledge statements into the prompt often yields limited gains.

Rather than retrieving and applying the knowledge based on the NL query, Tk-Boost first asks the NL2SQL agent to write a "first attempt" SQL query, and then retrieves and applies the knowledge to fix logical errors in this SQL. To enable accurate retrieval, Tk-Boost indexes the knowledge based on *applicability conditions* which specify, for each knowledge statement, features of SQL queries (e.g., columns used and SQL clauses present) that may benefit from the knowledge, that is, features of a SQL query wherein the misconception may be present. Then, at query time, Tk-Boost retrieves knowledge whose applicability conditions include the query features of the agent-generated SQL, uses this knowledge to identify misconceptions in the SQL query, and provides fine-grained feedback to the agent consisting of NL statements on how to correct errors in its SQL. Tk-Boost does so one-by-one for each sub-query within the agent-generated SQL, ensuring correct application of the feedback in complex queries. Fig. 1 illustrates this process. In Step 2, the applicability condition of a knowledge statement specifies that it applies to queries accessing the `brand` and `name` columns. In Step 3, for a new query, the agent initially generates a SQL query that incorrectly uses `brand`. After retrieving the relevant knowledge, Tk-Boost provides feedback indicating that `name` should be used instead, leading the agent to produce the correct SQL.

Contributions. Tk-Boost is a bolt-on framework to augment any NL2SQL agent with tribal knowledge, providing significant accuracy boosts for any NL2SQL agent by learning how to use a database through experience. Specifically, we:

- Present Tk-Boost, the first framework that enables any NL2SQL agent to accumulate knowledge of how to query a database accurately *from experience*;
- Introduce the notion of tribal knowledge for data agents to correct their misconceptions;
- Show how to reliably discover tribal knowledge from agent's past experience and use it to improve accuracy for future queries;
- Illustrate through extensive empirical evaluation that **Tk-Boost significantly improves a given NL2SQL agent's accuracy, by up to 16.9% on Spider 2.0 and 13.7% on BIRD benchmarks**. These accuracy boosts are up to **10% better than memory-based baselines** that provide modest gains of less than 5%.

2 BACKGROUND AND PROBLEM STATEMENT

2.1 NL2SQL Agents

NL2SQL Translation. We study the problem of answering natural language (NL) queries on a relational database. Let $\mathcal{D}$ denote a relational database on which a user issues a natural language query, q. Our goal is to generate a SQL query s that is equivalent to q. The answer to q is then the result of executing s on $\mathcal{D}$. We denote the result of executing s on $\mathcal{D}$ by ExecSQL$(s, \mathcal{D})$, which returns either an output relation or an error string describing an execution failure.

Translation with NL2SQL Agents. To perform NL2SQL translation, existing methods use LLMs to translate the given NL query to SQL [13, 14, 28, 30]. We refer to LLM-powered functions that generate SQL for an NL query as an *NL2SQL agent*, and denote it by $\mathcal{A}$. NL2SQL agents are often given access to the database, and query the database to explore the data before generating the final SQL query (they may additionally have access to other tools, e.g., reading text files, which we omit without loss of generality). Specifically, to perform NL2SQL translation, the agent is first provided with the NL query (along with additional prompts or metadata, omitted for simplicity). It then iteratively issues intermediate SQL queries to the database whose results are accumulated into a running *context* and supplied back to the agent at each subsequent iteration. The agent uses this context to generate new SQL queries, eventually generating the final SQL query after which it terminates.

To formalize this process, denote by C the space of all possible text strings and by $\mathcal{S}$ the space of all possible SQL queries. An NL2SQL agent is a function $\mathcal{A} : C \to \mathcal{S} \times \{0, 1\}$, implemented using an LLM, that takes a string as input and outputs a SQL query and a Boolean indicator, where the latter denotes whether the output SQL is the final translation result or an intermediary SQL. We refer to the input to the agent, $C \in C$, as its *input context* or *context* for short. To generate a SQL query for an NL query q, the agent is given the query q, and proceeds iteratively to generate the SQL query. We denote by C_t the context of the agent at the t-th iteration and therefore the *initial context* is $C_0 = q$. At the t-th iteration ($t = 0$ initially), the agent is given C_t as input and produces a SQL query, s_t along with a Boolean variable, `is_final`, that denotes if the query is the final translation result, i.e., s_t, `is_final` $= \mathcal{A}(C_t)$. s_t is then executed on the database to obtain $R_t =$ ExecSQL$(s_t, \mathcal{D})$. The output of the SQL query, R_t, as well as the query s_t is concatenated to the context, C_t, to create the context for the next iteration, $C_{t+1} =$ `concat`(C_t, s_t, R_t), where `concat` is a function that concatenates strings. In practice, C_t contains prompts as well as thinking tokens [39, 50] (and any other tool use by the agent) which we omit to simplify notation. This process repeats until the agent indicates it has generated the final SQL, s_n, at the n-th iteration with n denoting the total number of iterations. We refer to C_{n+1} as the agent's *execution trace*, which contains all intermediary steps, the final SQL generated and its output.

LLMs and Embedding Models. In addition to the NL2SQL agent, we assume access to an LLM and an embedding model, which can be used to help with translation. We use the embedding model to embed strings and compute their cosine similarity when performing similarity search; search between strings x and x' is denoted by `emb-sim`(x, x'). Additionally, the LLM is not an NL2SQL agent unlike $\mathcal{A}$ and does not have query access to the database.

Table 1: Notation.

Symbol	Description
Database and Query	
$\mathcal{D}$	Relational database
q	NL query
s	SQL query; $s^\star$ denotes ground-truth SQL
NL2SQL Agent and Augmentation	
t	Iteration index
$\mathcal{A}$	NL2SQL agent
C_t	Agent context at t
τ	Agent execution trace
$\mathcal{E}$	Experience set
M	Augmentation mechanism
C_t^M	Augmented context
Q	Dist. future NL queries

Algorithm 1: Agent Augmented with Mechanism M

```
1  C_0 ← q
2  t ← 0
3  C_0^M ← M(C_0, E)
4  is_final ← False
5  while is_final = False do
6      (s_t, is_final) ← A(C_t^M)
7      R_t ← ExecSQL(s_t)
8      C_{t+1} ← concat(C_t^M, R_t, s_t)
9      C_{t+1}^M ← M(C_{t+1}, E)
10     t++
11 end
12 return s_{t-1}
```

2.2 Augmenting Agents using Experience

Augmented Agents. Our goal is to improve the accuracy of an NL2SQL agent by allowing the agent to accumulate experience when using the database. We consider the setting where the agent is initially given a few NL query examples with known ground-truth to accumulate experience, analogous to a "training phase" for an LLM. These examples are used to help answer future queries through an *augmentation mechanism* that uses the experience to provide feedback to the agent. To formalize this process, first define an *experience tuple* as a tuple $(q, \tau, s^\star)$, where q is a natural-language query, τ is the agent's execution trace when attempting to answer q, and $s^\star$ is a SQL query that produces the ground truth result. We assume access to a collection of N past experience tuples, $\mathcal{E} = \{(q_i, \tau_i, s_i^\star); \forall i \in [N]\}$, called an *experience set*. We assume our experience set contains queries on typical tables and columns users query within the database as well as queries requiring typical (complex) SQL logic on which NL2SQL agents may make mistakes.

To use the experience set when generating a new query, we define an *augmentation mechanism* as one that that modifies the agent's context during execution based on $\mathcal{E}$. Formally, let C_t denote the agent's context at step t. An augmentation mechanism is a function, M, which takes as input the past experience $\mathcal{E}$ and a context C_t, and produces a new context C_t^M by optionally adding natural language feedback derived from $\mathcal{E}$ to C_t, i.e., $C_t^M = M(\mathcal{E}, C_t)$. Note that C_t^M can simply contain experiences tuples appended to C_t, or—more beneficially as we show later—can contain knowledge distilled from the experience. This new context C_t^M is then provided to the NL2SQL agent at step t instead of C_t, producing a SQL query s_t, `is_final` $= \mathcal{A}(C_t^M)$. We say the mechanism M *augments* the agent $\mathcal{A}$ when it is used to modify its context from C_t to C_t^M. Alg. 1 shows the process of using an augmented agent to perform translation. M modifies the agent's context (Line 7) and $\mathcal{A}$ is called on this modified input context (Line 3). We call an agent that follows Alg. 1 an *augmented agent*.

Problem Statement. Our goal is to design an augmentation mechanism that uses past experience to improve the accuracy of the NL2SQL agent on future queries. Formally, let Q be the distribution of future NL queries. Then, given an agent $\mathcal{A}$ and an experience set $\mathcal{E}$, our goal is to design a mechanism M that augments $\mathcal{A}$ (as in Alg. 1) to improve its accuracy on queries from Q. Let $s(q, M)$ denote the translation result $\mathcal{A}$ produces when augmented with a

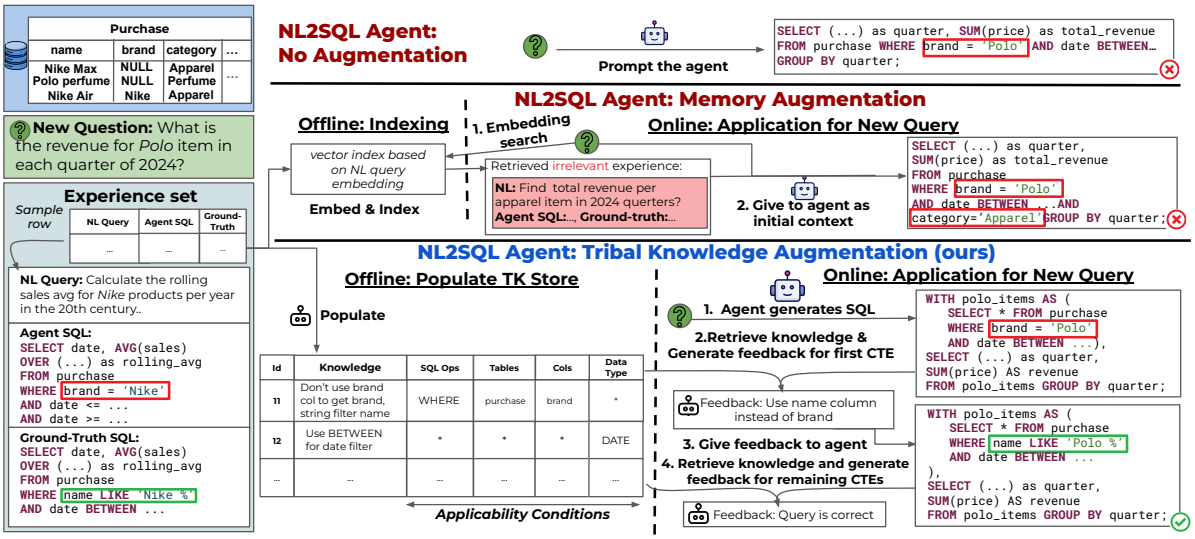


Figure 2: Example comparing Tk-Boost with other augmentation mechanisms.

mechanism M. Our objective is to design an augmentation mechanism M that maximizes execution accuracy on future queries:

$$\max_{M} \ \mathbb{E}_{q\sim\mathcal{Q}}\left[\mathbb{I}[\textsc{ExecSQL}(s(q,M),\mathcal{D}) \equiv \textsc{ExecSQL}(s^*,\mathcal{D})]\right],$$

where $\equiv$ denotes execution equivalence and $\mathbb{I}$ is indicator function.

3 AUGMENTING NL2SQL AGENTS WITH TRIBAL KNOWLEDGE

We augment the NL2SQL agent with *tribal knowledge* that enables the agent to accumulate and use knowledge derived from experience to correctly query the database. We first introduce a simple memory augmentation solution in Sec. 3.1 to motivate our design, present an overview of Tk-Boost in Sec. 3.2, with details in Secs. 4 and 5.

3.1 Naive Approach: Memory Augmentation

A simple augmentation mechanism is to include relevant experience tuples in the agent's context, similar to how memory stores do so through a process of RAG [20]. Given a query q, we retrieve past experience tuples for similar queries as

$$\mathcal{E}_q = \underset{(q_i,\tau_i,s_i^*)\in\mathcal{E}}{\text{top-k}} \ \texttt{emb-sim}(q_i, q),$$

where top-k is the k most similar experiences. We then use the augmentation mechanism M to include $\mathcal{E}_q$ in agent's initial context:

$$C_t^M = \texttt{concat}(q, e_1, ..., e_k) \text{ when } t = 0; \quad C_t^M = C_t \text{ otherwise,}$$

where $e_i = \texttt{concat}(q_i, \tau_i, s_i^*)$ corresponds to the i-th experience tuple (q_i, τ_i, s_i^*) in $\mathcal{E}_q$. To illustrate this approach, Fig. 2 revisits the example from Sec. 1: querying a database that contains a `purchase` table where `brand` is inconsistently populated and instead `name` reliably includes product names and brands (top left). The experience set $\mathcal{E}$ records a prior query where the agent incorrectly used `brand` instead of `name`. When answering a new query (highlighted green), the agent without any augmentation mechanism repeats this mistake [50]. Naive memory augmentation instead retrieves past experience tuples from a memory store (implemented as a vector index over query embeddings) and injects them into the agent's initial context, but this results in even more errors.

This approach has two main drawbacks. First, embedding similarity often retrieves irrelevant past experiences. In Fig. 2, the unaugmented agent's error stemmed from misunderstandings about which column to use, and the retrieved experience—though highly similar to the new question—failed to address the misunderstanding because it did not involve the `name` or `brand` columns that caused the error. Second, the agent often misuses retrieved experience appended to its initial context, drawing incorrect conclusions that introduce new errors. In Fig. 2, the agent wrongly adds a filter on the category column simply because it appeared in past experience.

3.2 Tk-Boost: Tribal Knowledge Augmentation

Past experience on its own is insufficient to correct the agent's logical errors when answering new queries. Instead, we propose to discover *tribal knowledge* from the past experience and provide this knowledge to the agent when pertinent. To do so, we propose Tk-Boost; Tk-Boost discovers tribal knowledge describing how to correctly query the database and stores it in a *Tribal Knowledge Store* (TK-Store) for retrieval. To use this knowledge, instead of inputting information in the models' initial context to *prevent* agent's mistakes as our naive approach does (Sec. 3.1), Tk-Boost uses *corrective augmentation*, i.e., using tribal knowledge to *correct* the agent's mistakes after they occur. That is, when the agent produces a SQL query s_t at the t-th iteration, the augmentation mechanism, M, evaluates s_t to find errors. If it finds errors, it augments the context C_t to provide feedback for correcting the SQL. Corrective augmentation enables accurate retrieval and feedback because it takes the agent-generated SQL into account, compared with the naive approach that uses only the original NL query.

Concretely, Tk-Boost consist of two phases: (1) populating TK-Store in an offline step and (2) using TK-Store to augment the agent for new queries. We describe the TK-Store interface and data model in Sec. 3.2.1, overview how it's populated in Sec. 3.2.2, and discuss its use to augment the agent in Sec. 3.2.4.

3.2.1 Tribal Knowledge Store. The Tribal knowledge Store (TK-Store) is a data structure that stores tribal knowledge and supports retrieval. Here, we describe its interface and data model. The implementation of its operations is discussed in Sec. 4.

Data Model. TK-Store stores tribal knowledge; for each such tribal knowledge statement in NL, TK-Store stores *applicability*

conditions, which are features of a SQL query for which the knowledge is expected to help correct misconceptions. We consider four SQL features: *SQL keywords* used in the query, *table names* and *column names* for tables and columns accessed, and *data type* of the columns accessed in the query. Applicability conditions are used to retrieve knowledge statements. Informally, a knowledge statement is retrieved if a given SQL query's features *match* the knowledge's applicability condition. A match occurs if the query contains the SQL keywords, tables, columns and/or data types included in the knowledge statement's applicability condition (details in Sec. 4.2).

Internally, knowledge statements and their applicability conditions are stored in a table. This table contains five columns: the knowledge statement, and the four features mentioned above. The knowledge statement is a string field, and the applicability conditions are stored as four columns each corresponding to a query feature, where each feature is stored as a set of strings.

Fig. 2 (bottom) shows TK-Store with two sample rows. The first row (id=11) contains knowledge about using `name` column instead of `brand` when filtering products and the second about using BETWEEN keyword instead of a string filter for date values. Both knowledge statements have associated applicability conditions. For the first row, the applicability condition specifies that this knowledge is applicable to queries containing a WHERE clause on `brand` column in the `purchase` table. The value $*$ in a column implies that column can be ignored when evaluating whether the knowledge is applicable to a SQL query. For the second row the applicability condition states that the knowledge is applicable to queries that access a column with a date format.

Interface. TK-Store supports two operations: Insert and Retrieve. Insert(k, a) takes a knowledge statement, k, and its applicability condition, a, as input and inserts it into the TK-Store by simply creating a new row in the table. Retrieve(s) is a function that takes as input a *SQL query*, s, and outputs a subset of the knowledge statements in the TK-Store that can help resolve misconceptions in s. The Retrieve operation takes a SQL query as input to support corrective augmentation—that is, the Retrieve function is used after the agent has already generated a first attempt SQL query which is used to retrieve relevant knowledge. Details of the Retrieve operations are presented in Sec. 4.2.

3.2.2 Populating TK Store. We define a function Populate$(\mathcal{E})$ that takes an experience set $\mathcal{E}$ as input, discovers tribal knowledge and applicability conditions based on the experience set, and inserts them into the TK-Store. It does so by observing the logical errors in the experience set and generating reusable knowledge statements to help correct them, detailed in Sec. 4.1. Fig. 2 shows an example where multiple rows are generated from an experience tuple. The agent-generated SQL in the experience tuple uses an incorrect column (`brand` instead of `name`) and the population process generates a knowledge statement specifying the correct column.

3.2.3 Scope of tribal knowledge. Fig. 3 shows the types of knowledge Tk-Boost is designed to capture, categorized along two axes: whether or not it is about the data and/or about the questions. Knowledge only about the data (bottom right, Fig. 3) or questions (top left, Fig. 3) respectively help agents better understand data and questions, for example, with information about data formatting issues or business logic or terminology. Knowledge about both

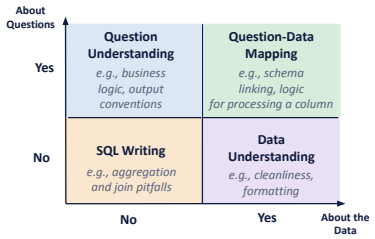


Figure 3: Types of knowledge Tk-Boost captures

data and questions discusses how to correctly map questions to data (e.g., schema linking for questions about a business sector). Finally, knowledge neither about the data nor questions contains general SQL writing tips (e.g., avoiding common join pitfalls). Although Tk-Boost can capture these knowledge types, in practice the captured knowledge depends on the experience set. In our experiments, most of the knowledge statements are from the bottom row and few about questions (top row, Fig. 3). A detailed taxonomy of the observed knowledge types in our experiments is provided in Sec. 6.5.

Finally, Tk-Boost assumes there is a single ground-truth interpretation for a question (based on the given experience set), and thus does not capture differing knowledge for different users or domains. For example, a given business term can have different meaning across companies (e.g. total revenue can mean quarterly or annual), and Tk-Boost, as currently designed cannot capture all such variations (they can lead to contradictory knowledge in the TK-store). We expect extensions to our knowledge store schema (e.g., to include domain, or user as an additional column) will allow Tk-Boost to handle such scenarios, which we leave to future work.

3.2.4 Augmentation Mechanism Overview. Using *corrective augmentation*, given a new query, Tk-Boost first asks the NL2SQL agent to generate a first attempt SQL to answer the query. Tk-Boost retrieves knowledge based on this first attempt. The retrieved knowledge is converted into NL feedback and provided to the NL2SQL agent to fix any potential mistakes in the query. Since SQL queries in practice typically contain multiple sub-queries, the NL2SQL agent often fails to correct errors if given feedback for the entire query at once. Instead, Tk-Boost provides feedback one sub-query at a time, allowing for more fine-grained steering of the NL2SQL agent (as described below).

To do so, Tk-Boost instructs the agent to use Common Table Expressions (CTEs) to write queries. It uses tribal knowledge to identify misconceptions, one CTE at a time. Per CTE, it retrieves relevant knowledge from the TK-Store, and based on the retrieved knowledge, the database, and the original CTE, Tk-Boost issues feedback to the NL2SQL agent. After the agent fixes the CTE, Tk-Boost moves to the next CTE, providing feedback to the NL2SQL agent until all the CTEs are corrected.

Fig. 2 (bottom right) shows an example of this process. First, the agent produces a SQL containing a CTE that incorrectly uses the column `brand`. Tk-Boost uses the CTE to retrieve knowledge from the TK-Store and generate feedback that the column `name` should be used instead of `brand` (Step 2). Tk-Boost then provides this feedback to the NL2SQL agent (Step 3), which then corrects the CTE. Tk-Boost finally studies the other sub-queries generated, retrieves

Algorithm 2: POPULATE($\mathcal{E}$)

```
foreach e ∈ E do
    K ← GetCorrectionStatements(e)
    foreach k ∈ K do
        (k, a) ← GenTKRow(q, D, k)
        TK-Store.insert(k, a)
    end
end
```

knowledge and observes that the query is correct, providing this feedback to the agent which returns the final correct query.

4 TK-STORE POPULATION AND RETRIEVAL

We now describe how TK-BOOST populates TK-Store from past experience sets and how TK-Store perform retrieval. We discuss TK-Store population in Sec. 4.1 and retrieval in Sec. 4.2.

4.1 TK Store Population

The POPULATE($\mathcal{E}$) function is responsible for extracting tribal knowledge and applicability conditions from the experience set $\mathcal{E}$ and populating the TK Store. A simple method is to directly ask an LLM to analyze each experience tuple, $(q, \tau, s^*) \in \mathcal{E}$ and identify important facts about the database or how to answer future queries correctly. However, directly extracting facts from each experience can result in (1) incorrect or trivial knowledge and (2) knowledge specific to q that cannot be reused. This is because the LLM often cannot detect logical errors in the agent-generated SQL, especially when it is complex and errors depend on subtle data characteristics.

Instead, TK-BOOST first identifies the logical errors and generates *correction statements*, NL statements that correct the logical errors, through an iterative process that explores the database and analyzes differences between the agent and ground-truth SQL. TK-BOOST then converts these correction statements into reusable knowledge. Each correction statement addresses a logical error specifically for the SQL query it was designed to fix; TK-BOOST identifies other queries where the same logical errors can occur and generates a knowledge statement to correct them, ensuring reusability. The applicability conditions are generated similarly, by identifying features of SQL queries where the logical errors may appear. This two-step process is shown in Alg. 2 (functions in blue are LLM pipelines; prompts are available in [1]).

4.1.1 Generating Correction Statements. To generate correction statements, we need to first find logical errors in the agent's SQL. Concretely, given an experience tuple, $e = (q, \tau, s^*)$, where s is the agent-generated SQL query (s is included in τ), we find the logical errors in s and generate correction statements that can correct these errors by comparing s with s^*. We generate separate correction statements for each logical error in s, generating *atomic* correction statements that can be combined together to correct different combinations of logical errors—we say a correction statement is *atomic* if it cannot be decomposed into multiple semantically meaningful statements that each address logical errors.

Algorithm. To accurately identify logical errors and generate correction statements, TK-BOOST first identifies a set of *candidate corrections* that are atomic and, together, address the logical errors in the SQL; from these candidate corrections, TK-BOOST removes incorrect or redundant corrections to obtain the final set of correction statements. While creating the candidate corrections set, we iteratively ask an LLM to explore the database and modify a single SQL clause (e.g., `WHERE` clause, `GROUP BY` expression) in the agent-generated SQL, by both producing a new SQL query and an NL statement that discusses the correction applied. We only allow single clause modifications to ensure each correction is atomic. This process continues until we obtain a SQL query that produces the correct answer, ensuring the logical errors in the SQL are fixed through this sequence of corrections. We use execution equivalence as a proxy for correctness, since testing logical equivalence of SQL queries is NP-hard in general [2, 38]. It is possible that execution equivalence test produces a false positive, which in turn can cause incorrect knowledge to be inserted in the TK-store. TK-BOOST attempts to prune out incorrect knowledge at application time, where LLM reasons about each retrieved knowledge statement's applicability to the current CTE and prunes inapplicable or incorrect ones (See Sec. 5). Our experimental results show TK-BOOST's knowledge application is robust to noisy knowledge (see Sec. 6.6.2).

Note that when generating correction statements, a single correction statement may not immediately result in execution equivalence. As a result, We repeatedly make corrections until we achieve execution equivalence, possibly after many iterations; at each iteration the output result of the query after the previous change as well as the ground-truth result are passed to an LLM and asked to design another modification to correct the SQL query. As we show in Appendix. B.4, in practice, for most questions this process takes multiple iterations but eventually generates a correct answer.

Nonetheless, not every correction made by the LLM may be correct or necessary. Thus, after all the corrections are performed, we make another LLM call to review its previous work and identify a subset of the candidate corrections that are needed to correct the logical errors following a self-reflection approach [42]. The self-reflection LLM call is given the candidate corrections, the agent-generated SQL, and the final SQL, and tags each correction as `necessary` or `unnecessary`: corrections needed to obtain the final SQL (e.g., changes to a CTE definition, join condition, or CTE-level predicate) are `necessary` and the rest are classified as `unnecessary`, with the decision left to an LLM. `unnecessary` corrections are then removed and knowledge is generated only from the `necessary` ones. Experiments in Appendix B.3.2 shows that this process is necessary able to prune out redundant or incorrect knowledge. Detailed pseudo-code is provided in Appendix A.1.

Example. Fig. 4 illustrates this process. Given the agent-generated SQL, TK-BOOST first generates a list of candidate correction statements to fix the query, done iteratively while exploring relevant tables and columns and taking the ground-truth SQL into account. Here, the LLM generates three candidate correction statements, one of which is to replace the `brand` filter with a string filter on `name` column—crucially, data exploration helps the LLM realize the `brand` column contains missing values while the `name` column reliably contains product brands. From the three candidate correction statements, the LLM removes the renaming of the column "quarterone" to "Q1" to create a final set of correction statements.

4.1.2 Generating Knowledge and Applicability Condition. For each correction statement created in the previous step, we generate

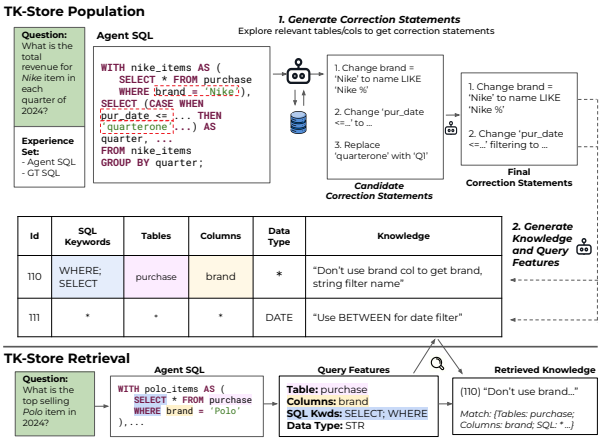


Figure 4: TK-Store population (top) and retrieval (bottom)

knowledge statements and applicability conditions jointly through a single LLM call to ensure they remain consistent.

Knowledge Statement. We instruct the LLM to use each correction statement, κ, to generate a knowledge statement that fixes the logical error, l, that the correction statement originally addressed. The knowledge generated is reusable, as it fixes l in all future queries where it may occur. In practice, this reusability is achieved by removing query specific details when possible, i.e., removing dependencies on column and table names, data types, or column values, so that the knowledge is applicable more broadly. Fig. 4 shows an example of this process where knowledge statements are created from correction statements (in Step 2), showing how the knowledge statements generalize correction statements. For Statement 1 in the final correction statements set, we see that the correction statement only applies to queries asking for Nike products, but knowledge generation creates a knowledge statement that applies to any query looking for product brands. Fig. 4 shows another example where the correction statement (Statement 2) specifies the column `pur_date`, but knowledge generation creates a more general statement that applies to any column with date information.

Applicability Conditions. An applicability condition for a knowledge statement specifies possible features of an incorrectly generated SQL where the agent has misconceptions. Using the applicability condition, retrieving the knowledge statement for future SQL queries with the same features becomes more reliable than semantic similarity search.

To obtain applicability conditions, the LLM is instructed to predict how a SQL query may be incorrectly formed for the target task, and to identify the features of the erroneous clauses—specifically, the SQL keywords involved, the tables and columns accessed, and the data types on which errors may occur.

Let $\mathcal{X} = \{$`SQL_keywords`, `tables`, `columns`, `data_types`$\}$ denote these four features. For each $x \in \mathcal{X}$, the LLM outputs either a list of values or a wildcard ($*$). Unless a wildcard is used, `SQL_keywords` must be valid SQL clauses (e.g., SELECT, WHERE, JOIN), `tables` and `columns` must exist in $\mathcal{D}$, and `data_types` must be valid SQL types (e.g., `int`, `date`). A feature is specified only if it must appear in an incorrect SQL; otherwise, a wildcard ($*$) is assigned.

Applicability conditions can specify when a knowledge statement applies, either to operations on particular tables or columns, or to operations on a specific data type. The first row in Fig. 4 illustrates the former, where the knowledge about using `name` instead of `brand` applies to queries that refer to the `brand` column. The second row illustrates the latter, where the knowledge applies to any operation involving a date column.

To ensure knowledge statements are generalizable, we construct an LLM prompt with example knowledge statements and provide three suggested generalization patterns to the LLM to help make the generalization more reliable. Specifically, our prompt encourages knowledge to follow one of the following patterns: (1) *knowledge about SQL operations on a specific table/column* (e.g., "use `name` not `brand`" on `purchase`) where applicability condition specifies wildcard for data type and values for other columns, (2) *knowledge about SQL operations on a specific data type* (e.g., "use `BETWEEN` for `date`-typed range filters") where applicability conditional specifies values for data type and wildcards for everything else, or (3) *general knowledge about SQL operations* (e.g., "include all composite-key components in `JOIN ON`") where the applicability condition specifies values for SQL Keywords and wildcards for everything else. We additionally encourage the LLM to ensure maximum generalizability within each category and place wildcard-values when unsure about specific applicability conditions. In Fig. 4, for Statement 1, the LLM follows pattern (1) and formulates the statement to maximally generalize to future queries over the same column, removing "Nike" value to enable better generalization to future queries over the column. In Statement 2, the LLM follows pattern (2), formulating a statement that maximally applies to queries over the same data type—to achieve which it removes column names.

Conflict Resolution. Newly proposed knowledge entries must be reconciled with existing ones before insertion into the TK-Store. Two entries are in conflict when their applicability conditions are identical or one is a strict superset of the other. In such cases, the entries are merged when both their applicability conditions are identical and their knowledge statements are semantically identical; otherwise the existing entry is retained and the new one discarded.

Store Maintenance. Each SQL-dialect maintains its own TK-Store, scoping engine-specific stylistic and dialect conventions to where they were learned. Furthermore, if multiple users are translating queries with TK-Boost, each user can be assigned its own TK-Store to prevent preferences of one user from impacting another; this can also be prevented by including the user's name as a part of applicability conditions for a knowledge statement. Doing so can also help prevent security and privacy issues in leaking a user's information to another.

4.2 TK Store Retrieval

At test-time, given a new query q and a SQL draft s, $\textsc{Retrieve}(s)$ returns the relevant knowledge statements that can help correct errors in s by first finding those whose applicability conditions *match* s, and then using an LLM to filter the matching statements and keeping only the statements needed to correct the mistakes.

Algorithm. To retrieve knowledge statements given a SQL query s, we first extract query features from s using a pipeline with a SQL parser, which extracts SQL keywords, tables, and columns that appear in the query s, and for each column we identify its data type by querying the database. The parser returns a dictionary c, such that $c[x]$ contains a set of values for each specific query feature $x \in \mathcal{X}$. Then, we retrieve knowledge statements by finding rows from the TK-Store whose applicability condition *matches* those of the query features. An applicability condition matches a

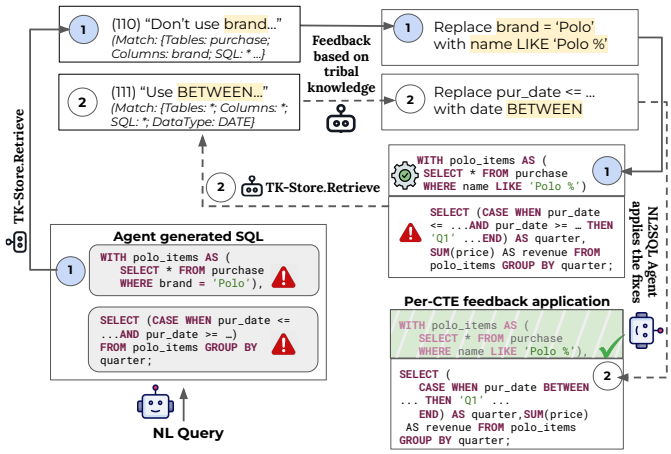


Figure 5: Knowledge application example.

query feature c if for each query feature, the applicability condition contains that feature or is a wildcard. Knowledge statements whose applicability condition matches the SQL query are filtered by an LLM to remove the knowledge statements deemed irrelevant. The pseudo-code is in Appendix A.2. Note that our retrieval approach is designed to capture SQL variations by using an *expanded* feature list. Specifically, in our knowledge store, each feature contains a list of values where an LLM has decided the knowledge statement applies—or a wildcard when the knowledge statement applies to any feature value—and an input query only needs to match one of the values in the list. This design leaves it to the LLM generating the applicability condition to decide what possible SQL variations to capture. For example, if an agent makes a mistake while using `ROW_NUMBER` to rank items (e.g., using the wrong column), the created knowledge entry can include both `ROW_NUMBER` and `RANK` in its applicability condition to ensure the knowledge is not deemed irrelevant on translation of a similar query.

Example. Fig. 4 illustrates how, given an SQL query, retrieval first identifies its query features and returns knowledge statements whose applicability conditions match those features. Here, the query features match the SQL keywords, tables, and columns of the first knowledge statement, and therefore the statement is retrieved.

5 AUGMENTING AGENTS USING TK STORE

Our augmentation mechanism specifies *when* to modify the agent's context as well as *how* to modify it. As discussed in Sec. 3.2, instead of using the experience set to modify the agent's initial context, Tk-Boost uses *corrective augmentation* that first asks the agent to generate a SQL query and uses the TK-Store to provide feedback to the agent to correct any errors.

Augmentation Mechanism. We modify the agent's context after it generates its first query attempt and provide feedback to ensure correctness. To detect whether the agent has produced a complete SQL query (the agent can issue intermediary SQL queries, as discussed in Sec. 2), the agent outputs a Boolean indicator, `is_final`, indicating query completion. The agent's context is modified only after `is_final=True`.

Given the SQL draft s, Tk-Boost retrieves knowledge for s and converts it into feedback f, which is appended to the agent's context. A key observation is that different sub-queries within a single SQL query may contain different misconceptions, and therefore correcting them requires different knowledge statements. To improve precision in retrieval and knowledge application, we retrieve knowledge and provide feedback one CTE at a time (the agent is instructed in its initial context to decompose its SQL into as many CTEs as possible). For simple translations, or when the agent produces a SQL with no CTEs, we treat the entire SQL as a single CTE. For each CTE, we first retrieve relevant knowledge and then use an LLM to summarize it into actionable feedback for CTE correction. Specifically, the LLM call returns NL instructions for correcting the CTE, which are appended to the agent's context. This process repeats until feedback has been provided for every CTE and the full SQL has been corrected. The pseudo-code for this algorithm is presented in Appendix A.3.

Example. Fig. 5 shows the augmentation process, where first the agent generates a SQL containing CTEs. Tk-Boost augments the agent by providing feedback on each CTE, one by one. For the first CTE, Tk-Boost retrieves relevant knowledge from TK-Store, stating that the `name` column should be used instead of `brand`. Tk-Boost then uses this knowledge to provide feedback to the NL2SQL agent about how to modify the CTE. The NL2SQL agent then applies the feedback, fixing the first CTE while the second CTE remains incorrect. Tk-Boost then moves on to the second CTE, now retrieving different knowledge from TK-Store, and producing feedback to modify how to filter the date column in the CTE. The NL2SQL agent applies this feedback and produces a correct CTE.

6 EVALUATION

We evaluate Tk-Boost as an augmentation mechanism for NL2SQL agents. In Sec. 6.1, we discuss the evaluation setup. End-to-end comparison with baselines are summarized in Sec. 6.2, and we additionally demonstrate Tk-Boost can augment various NL2SQL agents in Sec. 6.3. Furthermore, various ablation studies and sensitivity analysis are presented in Secs. 6.4- 6.6, including a knowledge taxonomy (Sec. 6.5.1), coverage analysis of our knowledge store (Sec. 6.5.2), and noise-robustness evaluation of our retrieval mechanism (Sec. 6.6.2). Appendix B presents additional sensitivity analysis, ablations and evaluations.

6.1 Setup

6.1.1 Tasks and Datasets. We evaluate Tk-Boost on two standard NL2SQL benchmarks, Spider 2 [19] and BIRD-mini-dev [23]. Spider 2 consists of three different subsets, each for a different database system (and dialects), specifically, SQLite, BigQuery, and Snowflake. We refer to these three subsets of Spider 2 as SQLite, BQ, and SF, respectively. We use a filtered BIRD subset that removes known annotation errors [18].

For each dataset, we use a randomly sampled 25% subset of queries to construct the experience set and evaluate on the remaining 75% (see Table 11 in Appendix B for exact test/train size). Experience sets are constructed independently for each dataset without experience or knowledge being shared. All experience and knowledge are derived exclusively from the training queries, and results are reported on held-out test queries. We evaluate all metrics as an average over two independent runs.

6.1.2 Baselines and Augmentation Mechanisms. For each experiment, we fix an NL2SQL agent $\mathcal{A}$ and compare Tk-Boost with different augmentation mechanisms that use the experience set

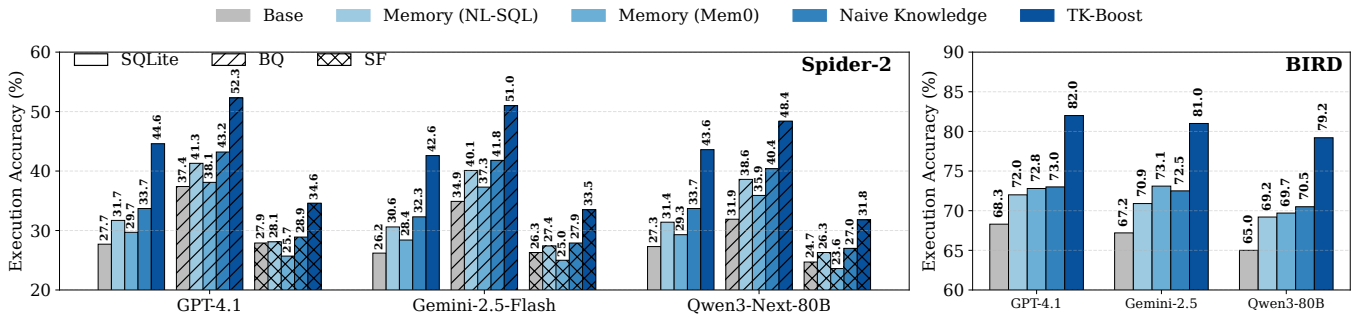


Figure 6: Execution accuracy across NL2SQL methods, (Left) Spider 2 and (Right) BIRD.

to improve the accuracy of $\mathcal{A}$. Each of our baselines has an augmentation mechanism that only modifies the *content* of the agent's context. Note that end-to-end comparison with state-of-the-art NL2SQL agents is done by applying Tk-Boost to those agents. We describe the agents used, including state-of-the-art NL2SQL agents Agentar-Scale-SQL-32B [46] and ReFORCE [11] in Sec. 6.1.3.

- **Base NL2SQL Agent.** The unaugmented agent A, corresponding to $M(\mathcal{E}, C_t) = C_t$.
- **Memory Augmentation.** The memory-augmentation mechanism described in Section 3.1.
  - **Mem0.** Mem0 [32] is used to augment the initial context C_0. It store agent execution traces from training queries. At test time, traces are retrieved using a cosine-similarity embedding search over NL queries.
  - **NL–SQL Pairs.** Instead of the entire execution trace as in Mem0, here only the NL question and its ground-truth SQL are provided. NL–SQL pairs are retrieved from the experience set using cosine similarity over NL-question embeddings.
- **Naive Knowledge.** An NL2SQL agent augmented with natural-language knowledge generated from NL–SQL pairs in the experience set with a single LLM call. Retrieved knowledge is injected into the initial context C_0 based on natural-language similarity, without explicit applicability conditions.
- **Tk-Boost (Ours).** This is our framework, as presented in Alg. 5.

6.1.3 Models and Agents.

Embeddings. In our main experiments, we perform similarity search using OpenAI `text-embedding-3-large` embedding model; the fuzzy retrieval ablation (Sec. 6.4.2) uses a SQL-aware encoder [37].

Models. Our evaluation uses four different LLMs: OpenAI GPT-4.1, Gemini-2.5-Flash, and Qwen3-Next-80B as well as Agentar-Scale-SQL-32B [46]. The first three span common pre-trained models used for NL2SQL [19, 27], and the fourth is a BIRD-specialized fine-tuned model—we use this model to show Tk-Boost can be used to augment agents even when they use models fine-tuned for a specific dataset. We use provider APIs for the closed-source GPT and Gemini models, and host the open-source Qwen model and Agentar-Scale-SQL-32B on two 80GB H100 GPUs.

NL2SQL Execution Frameworks. Unless otherwise stated, the NL2SQL function $\mathcal{A}$ (Sec. 2.1) follows a ReAct-style execution framework [50], where a single model iteratively generates SQL queries, executes them against the database, and refines them based on execution feedback. To evaluate the generality of Tk-Boost, we also instantiate $\mathcal{A}$ using ReFORCE [11], which replaces the single-model execution loop with a multi-component pipeline that includes schema compression, parallel SQL candidate generation, and majority voting, and uses multiple LLM calls.

6.1.4 Metrics. We report the following metrics:

- **Execution Accuracy.** Fraction of queries whose final SQL executes successfully and returns the correct result.
- **Robustness.** Fraction of queries answered correctly by the base NL2SQL agent but incorrectly after augmentation, measuring the risk of knowledge misapplication.
- **Latency and Turns.** Average end-to-end latency, and agent interaction rounds needed to produce the final SQL query, characterizing inference-time overhead.

6.1.5 Parameter Settings. Across all baselines, we use an uncapped inference budget with identical limits on the maximum number of agent turns. We retrieve top-K=5 items for approaches that use semantic similarity for retrieval, with K=5 chosen as it performed the best across baselines. All LLM calls use temperature t=0.5 and allow up to 128k tokens per run.

6.2 End-to-End Results

6.2.1 Execution Accuracy. Across models and datasets (see Fig. 6), **Tk-Boost yields accuracy gains of up to 16.9% on Spider-2 and 13.7% on BIRD**, representing a substantial improvement over the Base Agent. These gains significantly exceed the modest improvements from memory baselines (3.9%, 4.5% respectively) and naive knowledge baselines (5.8%, 4.7% respectively), and are significant strides in execution accuracy on these benchmarks, where gains of even 5% have historically required model finetuning or complex multi-agent pipelines with significant budget. In contrast, across all models, the Base NL2SQL Agent performs poorly with an execution accuracy of 24.7%–37.4% on Spider 2 and 65.0%–68.3% on BIRD, highlighting the difficulty of enterprise-scale text-to-SQL.

Memory-based augmentation yields only small gains (4.5%), primarily in cases where test questions closely match prior examples, particularly on datasets with many similar queries on a database (e.g., Spider 2 BigQuery). For most questions, memory retrieval surfaces irrelevant or misleading prior experiences, limiting generalization. The Naive Knowledge baseline shows modest gains by compressing past experience into generalized knowledge statements. However, this knowledge remains brittle, tied to surface-level query differences, and retrieval failures continue to occur.

Tk-Boost addresses these limitations in knowledge resuability by extracting tribal knowledge from past errors and defining structured applicability conditions for usage. As a result, it consistently improves across performance models.

6.2.2 Latency and Number of LLM Calls. We report Tk-Boost's online and offline inference cost on Spider 2 SQLite (GPT-4.1).

Table 2 reports online end-to-end latency and total number of LLM calls performed per query. Note that the function $\mathcal{A}$ is a ReAct agent that makes one LLM call per iteration; Tk-Boost makes

Table 2: Online # LLM calls and latency

Method	# LLM Calls	Latency (s)
Base NL2SQL Agent	12.0 ± 2.8	29.6 ± 9.2
Memory (NL–SQL Pairs)	13.6 ± 3.3	32.4 ± 10.1
Memory (Mem0)	14.5 ± 3.9	34.4 ± 10.8
Naive Knowledge	13.9 ± 3.6	33.2 ± 10.4
Tk-Boost (Ours)	15.9 ± 4.5	37.0 ± 13.3

Table 3: Online overhead based on # CTEs.

Draft SQL CTE Count	Base Agent	Tk-Boost overhead
0–1	10.7	+1.9
2–3	12.0	+4.4
4–8	14.1	+7.0

Table 4: Offline avg. # LLM cost per experience tuple

Pipeline Stage	Calls / Exp. Tuple	Share
Correction-statement generation	5.4	64%
Self-reflection	1.0	12%
Knowledge generation	1.0	12%
Applicability condition generation	1.0	12%
Total TK-Store Population Cost	**8.4**	**100%**

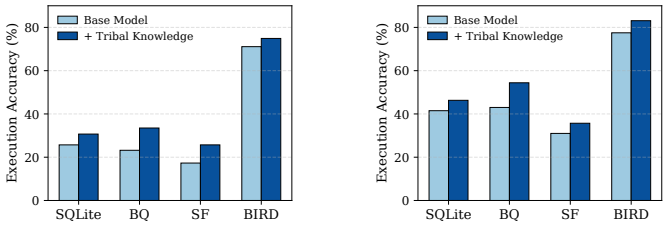
additional LLM calls for retrieval and feedback generation. The table shows Tk-Boost increases latency by 7.4 seconds over the base agent (29.6→37.0s). Total LLM calls rise modestly from 12.0 to 15.9 per query (+3.9). Online overhead is only due to CTE-level feedback refinement, which adds 2 LLM calls per CTE, so that Tk-Boost's per-query overhead grows with the number of CTEs in the agent's draft SQL. Table 3 shows this overhead based on the number of CTEs in the agent's draft, in terms of the number of additional LLM calls incurred by Tk-Boost, : +1.9 LLM calls for queries with 0–1 CTEs, +4.4 for 2–3 CTEs, and +7.0 for 4–8 CTEs. The base agent itself also uses more calls on complex queries (10.7 calls for 0–1 CTE queries, 12.0 for 2–3 CTEs, and 14.1 for 4–8 CTEs). Both Tk-Boost and the base agent scale linearly with CTE count, and Tk-Boost's per-query overhead is bounded at +7.0 LLM calls even in the highest-CTE bin. Online cost therefore scales with query complexity rather than with database size or TK-Store size.

Offline, TK-Store construction costs 8.4 LLM calls per training example for knowledge generation (Table 4): 5.4 on average for the iterative correction loop, and 1 call each for self-reflection, knowledge generation, and applicability condition generation. The latency in total is ≈21s per experience tuple and ≈12 min for the total knowledge generation (there are 34 samples in the experience set). Note that the total offline cost of $N \cdot 8.4 = 286$ LLM calls is paid once and further amortizes as more online queries are performed. Online cost scales linearly at $15.9 \times N_{\text{test}}$ calls, where N_{test} is the total number of queries performed online. The offline overhead as a fraction of online inference is therefore $286/(15.9 \times N_{\text{test}})$: 18% at $N_{\text{test}} = 101$ (the Spider 2.0 SQLite test set), and dropping to 7.2% at $N_{\text{test}} = 250$, 3.6% at $N_{\text{test}} = 500$, and 1.8% at $N_{\text{test}} = 1{,}000$.

6.3 Applicability to Other NL2SQL Agents

In Sec. 6.2, we discussed Tk-Boost's applicability to a ReAct agent with different base models. Here, we show it applies to specialized finetuned models and complex agent architectures that many of the best performing benchmark solutions utilize. We select solutions, namely Agentar-Scale-SQL [46] and ReFORCE [11] that achieve the highest execution accuracy on Spider 2 and BIRD, and have code available. Fig. 7 reports execution accuracy for each solution, where Tk-Boost is applied only at refinement time to improve generated SQL and does not interfere with the agent's SQL generation.

Although Agentar-Scale-SQL-32B performs strongly on BIRD (71.2%) as it is finetuned for it, it generalizes poorly to Spider 2 datasets. Applying Tk-Boost yields consistent gains across all datasets, improving accuracy on SQLite (25.3%→30.8%), BigQuery (23.4%→33.6%), Snowflake (17.2%→25.6%), and BIRD (71.2%→74.8%). These improvements indicate that Tk-Boost can identify and correct misconceptions even for an NL2SQL model that was already finetuned for the BIRD dataset [23]. Tk-Boost also improves ReFORCE, a strong multi-step agentic pipeline, raising execution accuracy from 41.5%→46.3% on SQLite, 43.0%→54.4% on BigQuery, and 31.0%→35.7% on Snowflake, and 77.5%→83.1% on BIRD.



(a) Agentar-Scale-SQL (32B) (b) ReFoRCE (GPT-4.1)

Figure 7: Augmenting existing NL2SQL solutions with Tk-Boost

6.4 Ablation Studies: Knowledge Type vs. Knowledge Retrieval and Application

We ablate tribal knowledge augmentation along two dimensions: (i) the *content* of the knowledge statements, and (ii) the *retrieval and application mechanism*. All experiments are conducted on Spider 2 SQLite using the same base NL2SQL agent.

6.4.1 Knowledge Type. Table 5 isolates the effect of knowledge content while keeping retrieval and CTE-level refinement fixed.

Both NL–SQL pair memory and naive knowledge improve execution accuracy (27.7%→32.7%, +5.0%; 32.7%→36.7%, +4.0%) yet fail to resolve logical errors, as they do not encode the underlying corrections. Correction statements, before generalizing into knowledge with applicability conditions, raise accuracy to 41.6% by capturing the required logical fix, but their sample-specific scope restricts reuse. Non-atomic corrections, unconstrained to a single clause-level edit, perform worse with an accuracy of 39.8%, as these corrections are broad rewrites that approximate the ground-truth SQL rather than capturing the agent's misconception.

Tribal knowledge achieves the highest accuracy at 44.6% by generalizing corrections and associating them with applicability conditions, enabling reusable fixes that encode both the required logical change and the contexts in which it should be applied.

6.4.2 Knowledge Retrieval and Application. Table 6 isolates the effect of retrieval and application while keeping knowledge content fixed to tribal knowledge statements. Additional ablation study evaluating importance of retrieval (comparing Tk-Boost with a no-retrieval setting) is provided in Appendix B.3.

Retrieving knowledge using NL-query similarity and injecting it before SQL generation achieves 31.7% accuracy. Applying the same knowledge during refinement on either the full SQL or on each CTE improves accuracy to 35.7% (+4.0%). This approach remains constrained by coarse retrieval that often doesn't retrieve knowledge that actually addresses the error. Retrieving knowledge from the generated SQL by structured applicability matching increases accuracy to 39.7% (+4.0%). Applying this knowledge for refinement on either the full SQL or each CTE yields similar accuracy (39.7% vs. 41.6%), indicating that gains at this stage arise primarily from improved retrieval. Retrieving and applying knowledge independently for each CTE achieves the highest accuracy at 44.6% (+4.9%), showing that knowledge is most effective when retrieval and application are used to identify and fix errors within each SQL sub-task

Table 5: Ablation of knowledge content

Knowledge Source	Acc. (%)
No knowledge (Base Agent)	27.7
Memory (NL–SQL Pairs)	32.7
Naive Knowledge	36.7
Non-Atomic Corr. Statements	39.8
Corr. Statements (Non-generalized)	41.6
Tribal knowledge (Tk-Boost)	**44.6**

Table 6: Ablation of knowledge usage. "–" means inapplicable.

Application Method	Retrieval Strategy		
	NL	SQL	CTE
NL-Level	31.7	–	–
SQL-Level	35.7	39.7	39.7
CTE-Level	35.7	41.6	**44.6**

at a finer granularity, rather than using coarse-grained feedback for broad suggestions on the entire SQL query.

We next ablate how we decide a query matches an applicability condition, where we consider relaxing our exact match requirement during knowledge retrieval. We evaluate two fuzzy retrieval baselines, both using the same SQL-aware encoder [37] (fine-tuned on SQL queries): (i) source-SQL embedding similarity, where each training query's source SQL is encoded and matched against the CTE being corrected; and (ii) TK-row embedding similarity, where each TK row (both knowledge statement and applicability conditions) is encoded and matched against the encoding of the features of the CTE being corrected. In both, a knowledge statement is retrieved if cosine similarity exceeds threshold τ. Fig. 8 reports accuracy of two fuzzy retrieval baselines, holding Tk-Boost's knowledge content fixed and varying similarity threshold τ. The highest accuracy reached by source-SQL embedding is 40.6% at $\tau = 0.6$, and by TK-row embedding is 40.6% at $\tau = 0.5$, both still 4.0% below Tk-Boost's **44.6**%. This result shows that using LLM's reasoning ability to decide the applicability conditions and retrieving with exact match on those conditions does better than relying on matching using embeddings similarity, even with SQL-aware embeddings.

6.5 Knowledge Taxonomy and Ablation

In this section we provide a taxonomy of the knowledge discovered by Tk-Boost through analysis of the errors produced by the base agent and the corresponding knowledge generated (Sec. 6.5.1. We also provide an ablation of knowledge type in the store based on this taxonomy (Sec. 6.5.2).

6.5.1 Knowledge Taxonomy. Here, we provide a detailed taxonomy of knowledge generated by Tk-Boost. To generate the taxonomy, we manually inspected knowledge in TK-store and agent errors they were derived from, and designed high level categories based on our observations, after which we tagged each knowledge statement and observed error with one of the categories. The high-level categories matches our discussion in Sec. 3.2.3, although here we provide more fine-grained categories based on our observations. The derived taxonomy is presented in Table 7. As tribal knowledge is designed to correct observed errors, each knowledge category corresponds to an NL2SQL agent error pattern, which is also shown in Table 7. The table additionally presents the number of knowledge statements in our TK-store for each category.

At the first level of our taxonomy, the knowledge statements are divided into two categories (analogous to the left and right halves of Fig. 3): **Database-Independent Knowledge** encompasses knowledge that informs the agent on how to perform SQL operations that it has previously struggled with (from data-usage misconceptions), where the errors are either due to incorrect SQL logic (e.g., incorrect join or aggregation operation) or a mismatch with expected output conventions. **Database-Dependent Knowledge** encompasses knowledge about specific tables, columns, or data types present within a database. For example, the agent makes recurring

Table 7: Taxonomy of tribal knowledge and error categories.

Category	# Stmts	Error Category
Database-Independent Knowledge		
Joins	16	Wrong join type, wrong join ordering
Windowing	16	Wrong `GROUP BY` level, aggregation at wrong granularity
Output conventions	25	`ROUND` precision, float-vs-integer output, alphabetical vs. logical sort order
Database-Dependent Knowledge		
Schema linking	10	Wrong table(s), wrong column(s)
Data dependent query logic	12	Operator behavior conditioned on specific data, e.g., `JOIN` key selection
Data cleanliness issues	18	Incorrect treatment of `NULL`s, duplicates or signed values, unit mismatches
Data value encoding	14	Incorrect treatement of tail values, categorical values, fiscal vs. calendar dates

mistakes due to data cleanliness (e.g., using an unusable column with many NULLs) or incomplete knowledge about data values (e.g., categorical values within a column) which are corrected with database-dependent knowledge. Note that each category in our taxonomy falls within one of the quadrants of knowledge types Tk-Boost is expected to capture, as discussed in Sec. 3.2.3.

An important knowledge category in our taxonomy is data cleaning (as was used in our running example), which often cause agents to use incorrect columns. Indeed, data cleaning issues are prevalent in our benchmarks, causing errors that Tk-Boost helps fix. We provide a more investigation on prevalence of data cleaning issues and how Tk-Boost helps solves them in Appendix B.4.3.

6.5.2 Ablation of Knowledge Types. To understand impact of different knowledge categories and the coverage needed in our experience set, we ablate Tk-Boost's accuracy based on different knowledge types. We evaluate the test accuracy when each knowledge type (based on our taxonomy in Sec. 6.5.1) is removed from the TK-store. For each knowledge type, we first remove corresponding knowledge statements from the full store, and then use Tk-Boost to answer test-set queries using only the remaining statements. Table 8 shows the results. Knowledge related to joins and data-cleanliness have the most significant impact on test-set accuracy, with a test-set accuracy of 31.7% and 33.7% respectively after removal.

6.6 Sensitivity Analysis

We analyze the sensitivity of Tk-Boost to (i) the size of the experience set used for knowledge discovery for different knowledge types, and (ii) the amount of noise contained in the training set. More experiments on sensitivity to other factors is presented in Appendix B.3. All experiments use GPT-4.1.

6.6.1 Experience Set Size. Table 9 reports execution accuracy as a function of the fraction, p, of the total number of Spider 2 queries (SQLite and Snowflake) used to construct tribal knowledge. On SQLite, accuracy rises from 29.7 at p=10% to 44.6 at p=25% (+14.9%), and further to 51.5 at p=50% (+6.9%). Snowflake shows a similar trend, improving from 27.4 to 34.6 (+7.2%) and to 39.7 (+5.1%). Most gains are realized by p=25%, with diminishing returns thereafter. Increasing the experience set size results in test-set accuracy increases on both SQLite and Snowflake, as more data misconceptions are captured when more experience becomes available.

We next study how many experience tuples are needed to obtain different types of knowledge, namely `database-independent` (`DI`, for short) and `database-dependent` (`DD`, for short) knowledge (we ablate across coarse grained categories as finer grained ones contain

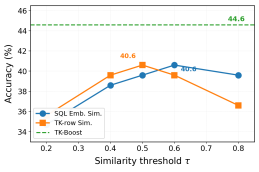


Figure 8: Ablation of applicability matching for retrieval.

Table 8: Test accuracy when ablating each knowledge category.

Knowledge Store	Store Size	Acc. (%)
Full Store	151	44.6
Database-Independent Knowledge		
– Joins	143	31.7
– Windowing	115	36.6
– Output conventions	112	37.6
Database-Dependent Knowledge		
– Schema linking	138	43.6
– Data dependent query logic	139	39.6
– Data cleanliness	133	33.7
– Data value encoding	126	37.6

too few knowledge statements). To isolate the impact of a specific knowledge type $X \in \{\texttt{DI}, \texttt{DD}\}$, we first remove all knowledge of type X from the TK-store, while keeping the rest of the knowledge intact. We then gradually reintroduce knowledge of type X into the TK-store based on the number of source queries they are derived from. That is, given a number of source queries i (we vary $i \in \{0, 4, 8, ..., 32\}$), we reintroduce only the knowledge of type X derived from the first i source queries.

Figure 9 shows the results of this experiment. We see that DI knowledge can be learned effectively from as few as 5 queries—while DD knowledge requires more queries. Intuitively, this is because knowledge about SQL operations generalizes across different databases, so fewer queries across disparate databases can provide the necessary knowledge. On the other hand, DD is database-specific, and knowledge for a specific database can only be learned from queries on that database. Due to the limited generalization of this type of knowledge, more queries are needed to improve accuracy.

6.6.2 Robustness to Noisy Training Data. We evaluate Tk-Boost's robustness to noisy training data on BIRD, which contains 55 queries with known annotation errors [18] (we refer to these as *noisy* queries; the remaining queries used for training are *clean*). We had filtered noisy queries out for our main evaluation; here we progressively re-introduce them. The training set always includes the 72 clean queries, and we add 0, 25, 50, 75, or 100% of the 55 noisy queries on top, yielding training sets that are 0%, 16%, 28%, 36%, or 43% noisy; the test set is held fixed at 211 clean queries throughout. Table 10 reports the test accuracy at different fractions of noise in the training set, as well as the base agent's accuracy for reference. Tk-Boost's accuracy drops only 1.8% (from 79.2% to 77.4%) as the noisy fraction grows from 0% to 43%, retaining a 9.1% margin over the base agent (68.3%) even at maximum noise. This robustness comes from Tk-Boost's feedback application mechanism (Sec. 5): knowledge whose applicability condition meets a query are retrieved and given to an LLM to summarize into per-CTE feedback, where the LLM typically decides not to apply incorrect knowledge, limiting the impact of any individual noisy correction statement.

7 RELATED WORK

NL2SQL Systems with Agents. Many recent NL2SQL systems adopt agentic designs that decompose SQL generation into iterative planning, tool use, and verification [7, 8, 12, 19, 22, 26, 33, 34, 40, 41, 48]. Some recent representative approaches are ReFORCE that introduces a multi-step agent architecture where candidate SQLs are generated, voted on, and refined for enterprise-scale databases [11], and AgenticData that augments agents with schema exploration, table sampling, and intermediate result inspection tools [43].

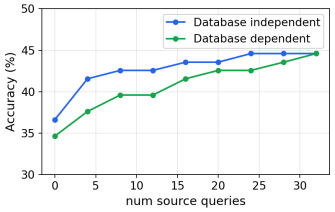


Figure 9: Impact of experience size by knowledge category.

	10%	25%	50%
SQLite	29.7	44.6	**51.5**
Snowflake	27.4	34.6	**39.7**

Table 9: Accuracy based on experience set size.

Table 10: Tk-Boost accuracy on BIRD as the fraction of noisy queries in the training set increases.

Noise ratio in Training Set	0%	16%	28%	36%	43%	*Base Agent (w/o Tk-Boost)*
Test Accuracy (%)	79.2	79.0	78.6	78.0	77.4	68.3

Tk-Boost operates orthogonally to these frameworks. Rather than modifying agent planning or tools, it acts as a bolt-on layer that provides feedback to the agents by discovering and using tribal knowledge. Given a draft SQL, Tk-Boost identifies recurring logical misconceptions and provides targeted feedback on subtasks. Our experiment shows that Tk-Boost improves accuracy of NL2SQL agents with varying modeling choices and agentic architectures.

Agentic memory. Prior work extends LLM agents with persistent memory to overcome context limits, including OS-style memory abstractions (e.g., MemGPT) [25] and approaches that extract salient information from interaction traces into persistent stores [9, 49, 54]. Simply retrieving past traces or summaries often reintroduces the same misconceptions, as they do not encode corrective knowledge about how to query the database [6, 47]. Tk-Boost instead distills corrective, reusable knowledge from past mistakes, which, significantly outperforms memory-based baselines.

Knowledge base generation for Text-to-SQL. Several NL2SQL systems incorporate external knowledge beyond the question and schema [4, 13, 15, 41, 55]. Knowledge-to-SQL generates database facts such as column semantics and value conventions to support SQL generation [4], but factual knowledge alone does not resolve logical misconceptions caused by data idiosyncrasies [17]. KAT-SQL builds an evolving NL2SQL knowledge base from training queries and schemas, retrieving top-k entries at query time [4, 53], similar to the Naive Knowledge baseline in our experiments. This knowledge is driven by surface-level patterns, captures syntactic rather than logical differences, and remains sensitive to semantic retrieval failures, even with ground-truth SQL supervision [4]. Tk-Boost departs from database facts and rule-based or constraint-driven SQL correction [36] by grounding knowledge in execution errors that encode how the database should be queried.

8 CONCLUSION

We introduced Tk-Boost, a bolt-on framework that augments NL2SQL agents with tribal knowledge distilled from execution mistakes. By storing knowledge with applicability conditions in a structured store and applying knowledge through fine-grained SQL refinement, Tk-Boost enables agents to learn how to use databases through experience. Experiments on Spider 2 and BIRD show up to 16.9% improvement in execution accuracy over baselines across models. Tk-Boost is robust and generalizes across agent architectures. Our findings suggest that structured corrective knowledge is necessary for improving data agents and advancing the path toward reliable natural-language access to enterprise databases.

REFERENCES

[1] Shubham Agarwal, Asim Biswal, Sepanta Zeighami, Alvin Cheung, Joseph Gonzalez, and Aditya Parameswaran. 2026. TK-Boost Source COde. https://github.com/AgentSmithLab/TK-Boost

[2] Alfred V. Aho, Yehoshua Sagiv, and Jeffrey D. Ullman. 1979. Equivalences among relational expressions. *SIAM J. Comput.* 8, 2 (1979), 218–246.

[3] Jinheon Baek, Horst Samulowitz, Oktie Hassanzadeh, Dharmashankar Subramanian, Sola Shirai, Alfio Gliozzo, and Debarun Bhattacharjya. 2025. Knowledge Base Construction for Knowledge-Augmented Text-to-SQL. arXiv:2505.22096 [cs.CL] https://arxiv.org/abs/2505.22096

[4] Jinheon Baek, Horst Samulowitz, Oktie Hassanzadeh, Dharmashankar Subramanian, Sola Shirai, Alfio Gliozzo, and Debarun Bhattacharjya. 2025. Knowledge Base Construction for Knowledge-Augmented Text-to-SQL. *arXiv preprint arXiv:2505.22096* (2025).

[5] Muhammad Imam Luthfi Balaka, David Alexander, Qiming Wang, Yue Gong, Adila Krisnadhi, and Raul Castro Fernandez. 2025. Pneuma: Leveraging llms for tabular data representation and retrieval in an end-to-end system. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.

[6] Asim Biswal, Chuan Lei, Xiao Qin, Aodong Li, Balakrishnan Narayanaswamy, and Tim Kraska. 2026. AgentSM: Semantic Memory for Agentic Text-to-SQL. *arXiv preprint arXiv:2601.15709* (2026).

[7] Asim Biswal, Liana Patel, Siddarth Jha, Amog Kamsetty, Shu Liu, Joseph E Gonzalez, Carlos Guestrin, and Matei Zaharia. 2024. Text2sql is not enough: Unifying ai and databases with tag. *arXiv preprint arXiv:2408.14717* (2024).

[8] Kaiwen Chen, Yueting Chen, Nick Koudas, and Xiaohui Yu. 2025. Reliable Text-to-SQL with Adaptive Abstention. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–30.

[9] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413* (2025).

[10] Yeounoh Chung, Gaurav T Kakkar, Yu Gan, Brenton Milne, and Fatma Ozcan. 2025. Is Long Context All You Need? Leveraging LLM's Extended Context for NL2SQL. *arXiv preprint arXiv:2501.12372* (2025).

[11] Minghang Deng, Ashwin Ramachandran, Canwen Xu, Lanxiang Hu, Zhewei Yao, Anupam Datta, and Hao Zhang. [n.d.]. Reforce: A Text-to-SQL agent with self-refinement, format restriction, and column exploration. In *ICLR 2025 Workshop: VerifAI: AI Verification in the Wild.*

[12] Ju Fan, Zihui Gu, Songyue Zhang, Yuxin Zhang, Zui Chen, Lei Cao, Guoliang Li, Samuel Madden, Xiaoyong Du, and Nan Tang. 2024. Combining Small Language Models and Large Language Models for Zero-Shot NL2SQL. *Proc. VLDB Endow.* 17, 11 (July 2024), 2750–2763. https://doi.org/10.14778/3681954.3681960

[13] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363* (2023).

[14] Orest Gkini, Theofilos Belmpas, Georgia Koutrika, and Yannis Ioannidis. 2021. An in-depth benchmarking of text-to-sql systems. In *Proceedings of the 2021 International Conference on Management of Data.* 632–644.

[15] Zijin Hong, Zheng Yuan, Hao Chen, Qinggang Zhang, Feiran Huang, and Xiao Huang. 2024. Knowledge-to-sql: Enhancing sql generation with data expert llm. *arXiv preprint arXiv:2402.11517* (2024).

[16] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. 2025. Next-generation database interfaces: A survey of llm-based text-to-sql. *IEEE Transactions on Knowledge and Data Engineering* (2025).

[17] Zezhou Huang, Shuo Zhang, Kechen Liu, and Eugene Wu. 2024. Data-Centric Text-to-SQL with Large Language Models. In *NeurIPS 2024 Third Table Representation Learning Workshop.*

[18] Tengjun Jin, Yoojin Choi, Yuxuan Zhu, and Daniel Kang. 2026. Pervasive Annotation Errors Break Text-to-SQL Benchmarks and Leaderboards. *arXiv preprint arXiv:2601.08778* (2026).

[19] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, et al. 2024. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. *arXiv preprint arXiv:2411.07763* (2024).

[20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[21] Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. 2024. The dawn of natural language to sql: Are we fully ready? *arXiv preprint arXiv:2406.01265* (2024).

[22] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2024. Codes: Towards building open-source language models for text-to-sql. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.

[23] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2024).

[24] Xiuwen Li, Qifeng Cai, Yang Shu, Chenjuan Guo, and Bin Yang. 2025. AID-SQL: Adaptive in-context learning of text-to-SQL with difficulty-aware instruction and retrieval-augmented generation. In *2025 IEEE 41st International Conference on Data Engineering (ICDE).* IEEE, 3945–3957.

[25] Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jihao Zhao, Yezhaohui Wang, et al. 2025. MemOS: An Operating System for Memory-Augmented Generation (MAG) in Large Language Models. *arXiv preprint arXiv:2505.22101* (2025).

[26] Shu Liu, Soujanya Ponnapalli, Shreya Shankar, Sepanta Zeighami, Alan Zhu, Shubham Agarwal, Ruiqi Chen, Samion Suwito, Shuo Yuan, Ion Stoica, et al. 2025. Supporting our ai overlords: Redesigning data systems to be agent-first. *arXiv preprint arXiv:2509.00997* (2025).

[27] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2024. A Survey of NL2SQL with Large Language Models: Where are we, and where are we going? *arXiv preprint arXiv:2408.05109* (2024).

[28] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2025. A Survey of Text-to-SQL in the Era of LLMs: Where Are We, and Where Are We Going? *IEEE Transactions on Knowledge and Data Engineering* 37, 10 (2025), 5735–5754. https://doi.org/10.1109/TKDE.2025.3592032

[29] Xinyu Liu, Shuyu Shen, Boyan Li, Nan Tang, and Yuyu Luo. 2025. Nl2sql-bugs: A benchmark for detecting semantic errors in nl2sql translation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2.* 5662–5673.

[30] Yuyu Luo, Guoliang Li, Ju Fan, Chengliang Chai, and Nan Tang. 2025. Natural language to sql: State of the art and open problems. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5466–5471.

[31] Kyle Luoma and Arun Kumar. 2025. Snails: Schema naming assessments for improved llm-based sql inference. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.

[32] Mem0. [n.d.]. Mem0 Memory Store. https://mem0.ai/

[33] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Sercan O Arik. 2024. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. *arXiv preprint arXiv:2410.01943* (2024).

[34] Ananya Rahaman, Anny Zheng, Mostafa Milani, Fei Chiang, and Rachel Pottinger. 2024. Evaluating SQL Understanding in Large Language Models. *arXiv preprint arXiv:2410.10680* (2024).

[35] Tonghui Ren, Yuankai Fan, Zhenying He, Ren Huang, Jiaqi Dai, Can Huang, Yinan Jing, Kai Zhang, Yifan Yang, and X Sean Wang. 2024. Purple: Making a large language model a better sql writer. In *2024 IEEE 40th International Conference on Data Engineering (ICDE).* IEEE, 15–28.

[36] Tonghui Ren, Chen Ke, Yuankai Fan, Yinan Jing, Zhenying He, Kai Zhang, and X. Sean Wang. 2025. The Power of Constraints in Natural Language to SQL Translation. *Proc. VLDB Endow.* 18, 7 (March 2025), 2097–2111. https://doi.org/10.14778/3734839.3734847

[37] s2593817. [n.d.]. sft-sql-embedding. https://huggingface.co/s2593817/sft-sql-embedding Hugging Face model card.

[38] Yehoshua Sagiv and Mihalis Yannakakis. 1980. Equivalences Among Relational Expressions with the Union and Difference Operators. *J. ACM* 27, 4 (Oct. 1980), 633–655. https://doi.org/10.1145/322217.322221

[39] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.

[40] Tobias Schmidt, Viktor Leis, Peter Boncz, and Thomas Neumann. 2025. Sql-storm: Taking database benchmarking into the llm era. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4144–4157.

[41] Chen Shen, Jin Wang, Sajjadur Rahman, and Eser Kandogan. 2025. MageSQL: Enhancing In-context Learning for Text-to-SQL Applications with Large Language Models. *arXiv preprint arXiv:2504.02055* (2025).

[42] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 8634–8652.

[43] Ji Sun, Guoliang Li, Peiyao Zhou, Yihui Ma, Jingzhe Xu, and Yuan Li. 2025. Agenticdata: An agentic data analytics system for heterogeneous data. *arXiv preprint arXiv:2508.05002* (2025).

[44] Zhaoyan Sun, Xuanhe Zhou, Guoliang Li, Xiang Yu, Jianhua Feng, and Yong Zhang. 2024. R-bot: An llm-based query rewrite system. *arXiv preprint arXiv:2412.01661* (2024).

[45] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. CHESS: Contextual Harnessing for Efficient SQL Synthesis. *arXiv preprint arXiv:2405.16755* (2024).

[46] Pengfei Wang, Baolin Sun, Xuemei Dong, Yaxun Dai, Hongwei Yuan, Mengdie Chu, Yingqi Gao, Xiang Qi, Peng Zhang, and Ying Yan. 2025. Agentar-Scale-SQL:

Advancing Text-to-SQL through Orchestrated Test-Time Scaling. *arXiv preprint arXiv:2509.24403* (2025).

[47] Xiangjin Xie, Guangwei Xu, Lingyan Zhao, and Ruijie Guo. 2025. Opensearch-sql: Enhancing text-to-sql with dynamic few-shot and consistency alignment. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–24.

[48] Guanming Xiong, Junwei Bao, Hongfei Jiang, Yang Song, and Wen Zhao. 2025. Multi-Turn Interactions for Text-to-SQL with Large Language Models. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 3560–3570.

[49] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110* (2025).

[50] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

[51] Haoxiang Zhang, Yurong Liu, Aécio Santos, Juliana Freire, et al. 2025. Autoddg: Automated dataset description generation using large language models. *arXiv preprint arXiv:2502.01050* (2025).

[52] Jiani Zhang, Zhengyuan Shen, Balasubramaniam Srinivasan, Shen Wang, Huzefa Rangwala, and George Karypis. 2023. NameGuess: Column name expansion for tabular data. *arXiv preprint arXiv:2310.13196* (2023).

[53] Kun Zhang, Xiexiong Lin, Yuanzhuo Wang, Xin Zhang, Fei Sun, Cen Jianhe, Hexiang Tan, Xuhui Jiang, and Huawei Shen. 2023. Refsql: A retrieval-augmentation framework for text-to-sql generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 664–673.

[54] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19724–19731.

[55] Jun-Peng Zhu, Boyan Niu, Peng Cai, Zheming Ni, Jianwei Wan, Kai Xu, Jiajun Huang, Shengbo Ma, Bing Wang, Xuan Zhou, et al. 2024. Towards Automated Cross-domain Exploratory Data Analysis through Large Language Models. *arXiv preprint arXiv:2412.07214* (2024).

Algorithm 3: GetCorrectionStatements(e)

Input : Experience tuple e
Output: Correction statements for logical errors in e

1 $(q, \tau, s^*) \leftarrow e$, $s \leftarrow$ incorrect SQL in τ
2 $\Delta \leftarrow \emptyset$
3 $R^\star \leftarrow \text{ExecSQL}(\mathcal{D}, s^\star)$, $R \leftarrow \text{ExecSQL}(\mathcal{D}, s)$
4 **while** $R \not\equiv R^\star$ **do**
5 $\quad (\delta, s) \leftarrow \text{MakeCorrection}(q, \mathcal{D}, s, s^\star, R, R^\star)$
6 $\quad R \leftarrow \text{ExecSQL}(\mathcal{D}, s)$
7 $\quad \Delta \leftarrow \Delta \cup \{(\delta, s, R)\}$
8 **end**
9 **return** GetRequiredCorrections(Δ)

Algorithm 4: Retrieve(s)

Input: SQL query s
Output: Applicable knowledge statements $\mathcal{K}_{\text{ret}}$

$c \leftarrow$ parse s and obtain query features
$\mathcal{K}_{\text{cand}} \leftarrow \emptyset$
foreach row r **in** TK-Store **do**
$\quad$ **if** $\bigwedge_{\forall x \in \mathcal{X}} \big((r[x] = *) \ \vee \ (r[x] \cap c[x] \neq \emptyset)\big)$ **then**
$\quad\quad \mathcal{K}_{\text{cand}} \leftarrow \mathcal{K}_{\text{cand}} \cup \{r[\texttt{knowledge}]\}$
$\quad$ **end**
end
/* **Context-aware filtering** */
$\mathcal{K}_{\text{ret}} \leftarrow \text{FilterKnowledge}(s, \mathcal{K}_{\text{cand}})$
return $\mathcal{K}_{\text{ret}}$

Algorithm 5: Agent Augmented with TK Store

Input: NL query q; database $\mathcal{D}$
Output: Final SQL query $\hat{s}$.

/* **NL2SQL Agent Loop** */
$C_0^{TK} \leftarrow q$; $t \leftarrow 0$; $i \leftarrow 0$
is_final = False
while is_final = False $\vee$ $f \neq \emptyset$ **do**
$\quad f \leftarrow \emptyset$
$\quad (s_t, \texttt{is\_final}) \leftarrow \mathcal{A}(C_t^{TK})$
$\quad$ **if** is_final = True $\wedge$ $i <$ number_of_CTEs_in(s_t) **then**
$\quad\quad$ /* **Get Feedback on i-th CTE Using TK Store** */
$\quad\quad f \leftarrow \text{GetTKStoreFeedback}(s_t, i, \mathcal{D})$
$\quad\quad i \leftarrow i + 1$
$\quad$ **end**
$\quad R_t \leftarrow \text{ExecSQL}(s_t, \mathcal{D})$
$\quad C_{t+1}^{TK} \leftarrow \texttt{concat}(C_t^{TK}, s_t, R_t, f)$
$\quad t \leftarrow t + 1$
end
return s_t

/* **Generate Feedback Using TK Store** */
Procedure GetTKStoreFeedback(s, i, $\mathcal{D}$):
$\quad (c_1, \ldots, c_L) \leftarrow$ List of all CTEs in s
$\quad \mathcal{K} \leftarrow$ TK-Store.Retrieve(c_i)
$\quad$ **return** Feedback(q, c_i, $\mathcal{K}$)

A TK-BOOST ALGORITHM DETAILS

Here, we provide the pseudo-code for the algorithms mentioned in the paper. Note that our source code containing detailed prompts is available at [1].

A.1 Generating Correction Statements

Here we provide the pseudo-code for correction statement generation algorithms. This algorithm is shown in Alg. 3. Given an experience tuple, we iteratively ask an LLM to perform a correction for a single clause in the SQL (Line 5). The LLM outputs a new SQL query and an NL description of the correction applied, δ. All the corrections (the NL statements along with the corresponding SQL draft) are kept in the set Δ (Line 7). Finally, we ask another LLM to evaluate the full set of corrections and keep only those required to fix the logical errors (Line 9).

A.2 Retrieval Algorithm

Alg. 4 describes the process for retrieving knowledge statements given a SQL query s. We first extract query features from s using a pipeline with a SQL parser (Line 3), which extracts SQL keywords, tables, and columns that appear in the query s, and for each column we identify its data type by querying the database. The parser returns a dictionary c, such that $c[x]$ contains a set of values for each specific query feature $x \in \mathcal{X}$. Then, the algorithm finds knowledge statements by finding rows from the TK-Store whose applicability condition *matches* those of the query features (Lines 4-8). An applicability condition matches a query feature c if for each query feature, the applicability condition contains that feature or is a wildcard (Line 6), that is, if

$$\bigwedge_{\forall x \in \mathcal{X}} \big((r[x] = *) \ \vee \ (r[x] \cap c[x] \neq \emptyset)\big)$$

Finally, all knowledge statements whose applicability condition matches the SQL query are filtered by an LLM alongside the full SQL query to remove the knowledge statements deemed irrelevant.

A.3 Augmentation Algorithm with Tk-Boost

Recall that we modify the agent's context after it generates its first query attempt and provide feedback to ensure correctness. We provide the pseudo-code for this process in Alg. 5.

To detect whether the agent has produced a complete SQL query (the agent can issue intermediary SQL queries, as discussed in Sec. 2), the agent outputs a Boolean indicator, `is_final`, indicating query completion. The agent's context is modified only after `is_final=True`, as shown in Alg. 5 Lines 8–10. Given the SQL draft s, Tk-Boost retrieves knowledge for s and converts it into feedback f, which is appended to the agent's context (Lines 17–20). We retrieve knowledge and provide feedback one CTE at a time (the agent is instructed in its initial context to decompose its SQL into as many CTEs as possible). For simple translations, or when the agent produces a SQL with no CTEs, we treat the entire SQL as a single CTE. For each CTE, we first retrieve relevant knowledge (Line 19) and then use an LLM to summarize it into actionable feedback for CTE correction (Line 20). Specifically, the LLM call returns NL instructions for correcting the CTE, which are appended to the agent's context (Line 13). This process repeats until feedback has been provided for every CTE and the full SQL has been corrected.

Table 11: Evaluation benchmarks and data splits. SQLite, BigQuery, and Snowflake are 3 datasets within Spider 2.

Datasets	Full	Train	Test
Spider 2 (SQLite)	135	34	101
Spider 2 (BigQuery)	205	50	155
Spider 2 (Snowflake)	208	29	179
BIRD	283	72	211

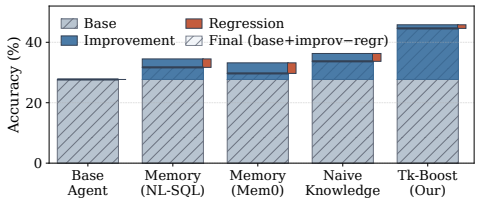


Figure 10: Accuracy decomposition on Spider 2 SQLite.

B ADDITIONAL EXPERIMENTS AND DETAILS

This section presents our robustness evaluation (Appx. B.1), analysis of error composition across methods (Appx. B.2), additional sensitivity and ablation studes (Appx. B.3), and empirical analysis of Tk-Boost's behavior and examples (Appx. B.4).

Table 11 provide details of the number test and training samples used in our experiments for each dataset.

B.1 Robustness Evaluation

We analyze the robustness of Tk-Boost and baselines in this section. To do so, we decompose their outcomes into *improvement* (the fraction of Base Agent errors that are corrected), and *regression* (the fraction of Base Agent correct predictions that become incorrect) after augmentation. Fig. 10 illustrates this on Spider 2 SQLite (GPT-4.1).

While memory-based methods yield >5% accuracy improvements before regressions, the net improvement over the Base Agent is limited (2.0% to 4.0%) as there are regressions of about 3% of initially correct questions. The high-recall nature of memory-based methods can mislead the agent by introducing irrelevant information from past experiences as purely semantic similarity is used for retrieval, irrespective of context. Using Naive knowledge achieves even larger raw improvements (8.6%), but regressions are still present (2.6%). Utilizing a singular LLM call to generate knowledge from an example NL–SQL pair introduces hallucination risk, as the LLM is not given context to understand why a mistake was made, and as a result, the knowledge misleads the agent when applied. We detail an example of this in Sec. 6.5.

Tk-Boost achieves both the greatest improvement and the least regression. It corrects 18.1% of the Base Agent's errors while only 1.2% of correct questions regress, yielding a net gain of +16.9%. Structured retrieval reduces errors occurring from retrieving irrelevant information, and as a result, the robustness is highest, with minimal regressions occurring due to ambiguous queries.

B.2 Error Composition

We analyze the percentage of errors that occur due to *data-selection* misconceptions versus *data-usage* misconceptions. After filtering out errors that occur due to agent runtime errors and instances

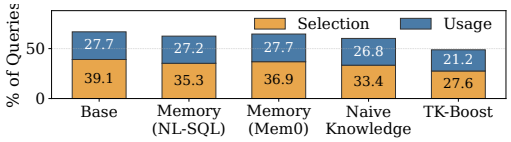


Figure 11: Error composition on Spider 2 SQLite (GPT-4.1) for each baseline. Tk-Boost significantly reduces data usage errors while also reducing data selection errors.

Table 12: Sensitivity analysis of # knowledge statements applied. for Tribal Knowledge (T3).

	Zero	All	Our
SQLite	27.7	35.6	**44.6**
Snowflake	27.8	30.2	**34.6**

where the agent does not produce a SQL, we utilize a SQL parser to extract tables and columns from each incorrect agent SQL and the corresponding ground truth SQL. If a mismatch is identified, we label it as a data-selection error, and otherwise we label the error as data-usage. We note that in scenarios where data-usage errors occur alongside data-selection errors, we consider data-selection as the main error source.

Fig. 11 illustrates the percentage of errors of each type across methods. Memory-based augmentation struggles to resolve any data usage error while providing a modest reduction in data selection errors (39.1% → 35.3%). Naive knowledge augmentation reduces data selection errors further (33.4%) but provides minimal resolution on data usage errors. While prior methods can reduce data-selection errors, data-usage misconceptions remain a substantial issue. Recalling examples directly from the experience set may reveal which tables and columns are appropriate for similar questions on a complex database, but the way to utilize the data is much harder to gather from direct memories or one-shot LLM-generated naive knowledge. In contrast, Tk-Boost reduces data-selection errors from 38.0% to 27.6% and data-usage errors from 28.0% to 21.2%, yielding a 15.9% improvement in execution accuracy overall.

B.3 Ablation and Sensitivity Analysis

We present additional ablation and sensitivity analysis results in this section.

B.3.1 Number of Retrieved Knowledge Statements. Here, we evaluate the sensitivity of Tk-Boost to the number of retrieved knowledge statements. We compares three settings: no knowledge, providing the full contents of the knowledge store, and retrieval using Tk-Boost's retrieval function.

The results of this experiment is shown in Table12. On SQLite, execution accuracy improves from 27.7 to 35.6 when all knowledge is applied (+7.9%), and further to 44.6 with scoped retrieval (+9.0% over all knowledge). On Snowflake, accuracy increases from 27.8 to 30.2 (+2.4%) with all knowledge and to 34.6 (+4.4%) with scoped retrieval.

The all-knowledge setting exposes the model to the entire knowledge store extracted from the 25% training split (80 knowledge statements for SQLite and 104 for Snowflake). In contrast, scoped retrieval with Tk-Boost selects knowledge per CTE, typically retrieving 2–8 statements conditioned on the query context (average

Table 13: Number of Iterations in Algorithm 3.

Metric	Spider 2.0 SQLite ($n = 34$)
Avg. atomic corrections per training run	5.4
Median atomic corrections per training run	4
% runs needing >1 atomic correction	93%

Table 14: Sensitivity to the generalization patterns on Spider 2.0 SQLite (GPT-4.1)

Setting	Acc. (%)	Imp. (%)	Reg. (%)	Δ vs Base
Base Agent	27.7	–	–	–
Tᴋ-Boost w/o generalization patterns	**42.6**	**16.5**	**1.6**	+14.9
Tᴋ-Boost (full)	44.6	18.1	1.2	+16.9

of 4.2). Across both backends, scoped retrieval consistently outperforms applying all knowledge, indicating that selective, context-aware retrieval is critical for effective knowledge application.

B.3.2 Ablation of Self-Reflection . On Spider 2.0 SQLite, the self-reflection step labels 29% (264/908) of correction statements as `unnecessary`. We furthermore evaluated removing the self-reflection LLM call, so that all the 264 `unnecessary` correction statement are used to generate knowledge statement. We see that TK-Boost's accuracy drops from **44.6**% to 40.6% (−4.0%), confirming that retaining the `unnecessary` statements would mislead the agent at test time.

B.3.3 Sensitivity to Generalization Patterns. Here, we evaluate the impact of removing the instructions about generalization patterns specified in Sec. 4.1 in the prompt. The results on Spider 2.0 SQLite (GPT-4.1) are shown in Table 14. Removing the three generalization patterns drops Tᴋ-Boost from 44.6% to **42.6**%, with improvement falling from 18.1% to 16.5% and regression rising from 1.2% to 1.6%. Overall, the results show that our guidelines do improve knowledge generation, but LLMs are still able to generate generalizable knowledge without them.

B.4 Tᴋ-Boost Analysis

B.4.1 Number of Correction Iterations. When generating correction statements in Alg. 3, a single correction statement may not immediately result in execution equivalence. As a result, Alg. 3 repeatedly makes corrections until it achieves execution equivalence, possibly after many iterations, where at each iteration the output result of the query after the previous change as well as the ground-truth result are passed to an LLM and asked to design another modification to correct the SQL query. To evaluate the reliability of this process, we analyze how often multiple atomic changes are performed to achieve execution equivalence. Our results show that Algorithm 3 takes 5.4 iterations on average over $n = 34$ training queries—each designing an atomic correction—to obtain execution equivalence, and that 93% of queries require more than one iteration but eventually reach execution equivalence (Table 13).

B.4.2 Knowledge Generalization. We evaluate the share of retrieved knowledge corresponding to each knowledge type. Specifically, we compute the average proportion of knowledge statements retrieved per test query that originate from training queries on the same database as a measure of cross-database knowledge transfer. We find only 8% of retrieved statements come from the same database; the remaining 92% are knowledge statements generated from other databases that match the test query showing that cross-database knowledge transfer is very common in practice.

Table 15: Prevalence of dirty-data queries across test sets, and Tᴋ-Boost's recovery. *Wrong in Base*: of dirty-data queries, those the base agent answers incorrectly. *Fixed by Tᴋ-Boost*: of those wrong in base, those Tᴋ-Boost answers correctly.

Dataset	Matched	Wrong in Base	Fixed by Tᴋ-Boost
Spider 2.0 SQLite	13 (12.9%)	11 (85%)	7 (64%)
Spider 2.0 BigQuery	20 (12.9%)	14 (70%)	6 (43%)
Spider 2.0 Snowflake	34 (19.0%)	28 (82%)	5 (18%)
BIRD	30 (14.2%)	14 (47%)	8 (57%)

Table 16: Representative tribal knowledge statements, grouped by misconception type.

Type	Tribal Knowledge Statement
Data-Selection	Filter calendar-year queries using date fields (e.g., `sales.date`), not fiscal-year attributes. Use entity-level metrics (e.g., business identifiers) for aggregation not surrogate join keys. Use `DISTINCT order_id` to define the entity when selecting records for order-level metrics.
Data-Usage	Compute window functions over the full date range; apply filters only after window evaluation. Validate join cardinality before aggregation; unintended many-to-many joins inflate results. Convert epoch fields to timestamps before applying date predicates.

B.4.3 Data Quality Issues. Here, we study what portion of the queries in the benchmark require handling data cleaning issues. To identify dirty-data queries, we use Claude Code with Opus 4.7 to decide whether each test query is uses dirty data or not—we provided a set of known dirty data issues we observed from our manual inspections as examples for the model to follow, e.g., handling partially populated columns (similar to our running example in Fig. 1), data with multiple encoding variants, case/whitespace normalization. To validate Claude Code's output, one of the authors manually verified its labels on a random 25% sample of the classified queries, finding 88% agreement. The results are shown in Table 15, where we see 21–25% of test queries across the four benchmarks require handling data cleaning issues. We also see that the base agent fails at 52–78% of the queries, and Tᴋ-Boost is able to correct 23–62% of those failures.

B.4.4 Representative Tribal Knowledge Statements. Table 16 shows representative tribal knowledge statements learned by Tᴋ-Boost that encode reusable corrections for each error type. For example, a Spider 2 SQLite query `local077` asks for three-month rolling averages and lagged values over 12 months on bank sales data; the agent applies a date filter before computing rolling averages and `LAG` values. However, this truncates the historical prefix required for window evaluation and produces biased aggregates. NL-similarity memory retrieves a "monthly revenue average" query that does not use window functions or `LAG`, and therefore only shows a plain `AVG` after filtering. Naive knowledge contains a broad instruction to "use WINDOW to compute window function" but omits the required ordering nuance; window functions must first be computed over the full date range before date filtering–so the same error repeats.

Tk-Boost instead provides the first data-usage rule in Table 16– compute window functions over the full date range and apply filters after–to the agent during refinement of the CTE, where it creates the window, precisely resolving this error while keeping other parts of the query untouched.